

Papers Dock



PAPER 4

PRACTICE PAPER 2

Open the document **evidence.doc**.

Make sure that your name, centre number and candidate number will appear on every page of this document. This document will contain your answers to each question.

Save this evidence document in your work area as:

evidence_ followed by your centre number_candidate number, for example:
evidence_zz999_9999

A class declaration can be used to declare a record.

A list is an alternative to an array.

1 A program stores the following 10 numbers in a 1D array with identifier numData.

10 4 23 45 12 8 21 11 3 1

(a) Write program code for a **new program** to:

- declare the global 1D array, numData, with 10 elements
- initialise numData in the main program using the data values shown

Save your program as **question1**.

Copy and paste the program code into part 1(a) in the evidence document.

[2]

(b) (i) A function BubbleSort() takes numData as by reference parameter and sort the elements in the numData array in ascending order.

Write program code for the function BubbleSort() and make sure the code is efficient.

Save your program.

Copy and paste the program code into part 1(b)(i) in the evidence document.

(ii) Test your program by calling the function BubbleSort().

Take a screenshot to show the result

Save your program.

Copy and paste the screenshots into part 1(b)(ii) in the evidence document.

[1]

(c) The procedure `BinarySearch()` takes the array and the value to be searched as Parameter and returns True if the value is in the array and False if the value passed as parameter is not in the array.

(i) Write program code for the procedure `BinarySearch()`.

Save your program.

Copy and paste the program code into part 1(c)(i) in the evidence document.

[6]

(ii) Edit the main program to:

- first call the procedure `BubbleSort()` so the array is sorted
- allow the user to input a number
- pass the value and array `numData` to `BinarySearch()` as the parameter
- output an appropriate message to tell the user whether the search value was found or not.

Save your program.

Copy and paste the program code into part 1(c)(ii) in the evidence document.

[5]

(iii) Test your program with one value that is in the array and one value that is not in the array.

Take a screenshot to show the result of each test.

Save your program.

Copy and paste the screenshots into part 1(c)(iii) in the evidence document.

[2]

2 A vending machine allows users to insert coins to purchase an item.

The user then enters the code for the item they would like the machine to dispense (give out). The user must re-enter the code until it is valid.

If the code is valid but the user has not inserted enough money for the item chosen, the machine waits for more coins to be inserted. The user then has to re-enter the code.

The user can press cancel at any time to return the money inserted into the machine.

The vending machine is part of a program that is written using object-oriented programming (OOP). The vending machine makes use of two classes that are described in the following tables.

All attributes are declared as private.

foodItem	
name : STRING	// the name of the item of food
code : STRING	// the code to be entered for that item to be // selected
cost : REAL	// the cost of the item
constructor(nameP, codeP, costP)	// creates an instance of foodItem // takes the name, code and cost as parameters
getCode()	// returns the code for the item
getCost()	// returns the cost of the item
getName()	// returns the name of the item

vendingMachine	
items : ARRAY[0:3] OF foodItem moneyIn : REAL	// stores four items of type foodItem // stores the total money inserted by the // user, initialised to 0 in the constructor
constructor(item1, item2, item3, item4)	// creates an instance of vendingMachine, // takes four objects of type foodItem as // parameters and stores them in array items
insertMoney()	// takes the value of the coin as a parameter // and adds it to moneyIn
checkValid ()	// takes a code as a parameter and checks it is // valid against the food item codes
getItemName ()	// takes the array index as a parameter and // returns the name of the food items

(a) The food Item is part of a program that is written using Object Oriented Programming.

All attributes are declared as private.

(i) Write **program code** to declare the class `foodItem`. You are only required to write program code for the attribute declarations and the constructor.

If you are writing in Python, include attributes declaration using comments.

Use your Programming languages's constructor method.

Save your program.

Copy and paste the program code into part 2(a)(i) in the evidence document.

[4]

(ii) Write the program code to add the methods `getCode()`, `getCost`, `getName()` in the `foodItem` class.

Save your program.

Copy and paste the program code into part 2(a)(ii) in the evidence document.

[3]

(b)(i) Write **program code** to declare the class `vendingMachine`. You are only required to write program code for the attribute declarations and the constructor.

If you are writing in Python, include attribute declarations using comments.

Use your programming language's constructor method.

Save your program.

Copy and paste the program code into part 2(b)(i) in the evidence document.

[4]

(ii) The method `checkValid()` takes the food item code as a parameter. It checks the code against each element in `items` and returns:

- -1 if the code is not valid
- -2 if the code is valid, but the `moneyIn` is less than the cost of the item
- the index of the item, if the code is valid and the `moneyIn` is greater than or equal to the cost of the item.

Write **program code** for the method `checkValid()`.

Save your program.

Copy and paste the program code into part 2(b)(ii) in the evidence document.

[5]

First Create the 4 food items

Item 1 : chocolate (**name** : "Oreo" **code** : 54 **cost** : 5.4)

Item 2 : sweets (**name** : "Custard" **code** : 12 **cost** : 2.1)

Item 3 : sandwich (**name** : "Peri Peri" **code** : 67 **cost** : 10.1)

Item 4 : apple (**name** : "green" **code** : 3 **cost** : 1.3)

(iii) Four objects of type `foodItem` are declared with the identifiers:

`chocolate, sweets, sandwich, apple`

Write **program code** to declare an instance of `vendingMachine` with the identifier `machineOne` and the objects: `chocolate, sweets, sandwich, apple`.

Save your program.

Copy and paste the program code into part 2(b)(iii) in the evidence document.

[4]

(iv) Amend the class `vending machine` and add the method `getItemName()` after that print the name of item on the 2nd index position.

Save your program.

Copy and paste the program code into part 2(b)(iv) in the evidence document.

[3]

(v) Take the screenshot of the result in (b)(iv)

Save your program.

Copy and paste the screenshots into part 2(b)(v) in the evidence document.

[2]

3) An Ordered linked list uses a 1D array to store the data. Each item in the linked list is of a record type, node, with a field data and a field nextNode.

The current contents of the linked list are:

startPointer	0	Index	data	nextNode
		0	1	1
emptyList	5	1	5	4
		2	12	7
		3	41	-1
		4	8	2
		5	0	6
		6	0	8
		7	32	3
		8	0	9
		9	0	-1

(a) The following is pseudocode for the record type *node*.

```
TYPE node
    DECLARE data : INTEGER
    DECLARE nextNode : INTEGER
ENDTYPE
```

Write program code to declare the record type *node*.

Save your program.

Copy and paste the program code into part 3(a) in the evidence document.

(b) Write program code for the main program.

Declare a 1D array of type node with the identifier `linkedList`, and initialise it with the data shown in the table on **page 2**.

Declare the pointers

Save your program.

Copy and paste the program code into part 3(b) in the evidence document.

[4]

(c) The procedure **outputNodes()** takes the array and `startPointer` as parameters. The procedure outputs the data from the linked list by following the `nextNode` values.

(i) Write program code for the procedure **outputNodes()**.

Save your program.

Copy and paste the program code into part 3(c)(i) in the evidence document.

[6]

(ii) Edit the main program to call the procedure `outputNodes()`.

Take a screenshot to show the output of the procedure `outputNodes()`.

Save your program.

Copy and paste the Screen Shot into part 3(c)(ii) in the evidence document.

[1]

(d) The function, `AddNode()`, takes the linked list and pointers and the value to be added as parameters.

Updates the pointers and returns `True` if it was Added successfully and `False` if the linked list is full

(i) Write program code for the function `AddNode()`

Save your program.

Copy and paste the program code into part 3(d)(i) in the evidence document.

[7]

- (ii) Edit the main program to:
- call AddNode() with the value 67
 - output an appropriate message depending on the result returned from AddNode()
 - call outputNodes() twice; once before calling AddNode() and once after calling AddNode().

Save your program.

Copy and paste the program code into part 3(d)(ii) in the evidence document.

[3]

- (iii) Test your program by Adding the data value 67 and take a screenshot to show the output.

Save your program.

Copy and paste the screenshots into part 3(d)(iii) in the evidence document.

[1]